EduMatrix Intelligence Platform

Comprehensive Technical Reference: System Architecture, AI/ML Models, DevOps Decisions, ATS Engine & Vector Databases

1. Authentication, Authorization & Security Architecture

EduMatrix enforces enterprise-grade security through **Role-Based Access Control (RBAC)** supporting 5 distinct roles: *Student, Lecturer, HOD, Placement Officer, and Admin*.

- **Password Hashing (Bcrypt)**: Plaintext passwords are never stored. The system utilizes passlib with the **Bcrypt** algorithm (backend/app/core/security.py), which applies a cryptographic salt and adaptive cost factor making rainbow table and brute-force attacks computationally impossible.
- **Irreversible One-Way Function**: Unlike two-way encryption (which can be decrypted if a master key is compromised), hashing is mathematically irreversible. Even database administrators cannot view passwords.
- **Storage Location**: Hashed strings with embedded salt are stored in the **PostgreSQL** users table inside the hashed_password column.
- **Stateless Authentication (JWT)**: Authenticated sessions issue cryptographically signed **JSON Web Tokens (JWT)** containing user ID, email, and role claims. This removes the need for memory-heavy server-side session tables.
- **Client State Management**: Tokens and user profiles are managed via **Zustand** store on the React client with automatic Axios authorization header injection.

2. Circulars, Documents & Auto-Extraction Engine

When a lecturer or HOD uploads an announcement or assignment as a **PDF, Image (PNG/JPG), or DOCX**, the system automatically parses content and extracts future deadlines:

- **Multi-Format Ingestion**: Handled via backend/app/api/upload.py and backend/app/api/circulars.py.
- **Digital vs Scanned OCR**: Digital PDFs are extracted in milliseconds via pdfplumber; scanned images and camera captures activate **Tesseract OCR with OpenCV CLAHE preprocessing**.
- **Future Date Detection**: The DateExtractor (backend/app/services/date_extractor.py) isolates dates (e.g. *'Submit on or before 28-08-2026'*) and filters for future deadlines.
- **Zero-Effort Form Auto-Fill**: Content, title, suggested type (Exam/Assignment/General), and due date automatically populate into the React UI (Circulars.jsx).
- **Object Storage**: Physical files are saved in high-performance **MinIO S3 Object Storage** rather than bloating PostgreSQL with raw binary blobs.

3. 5-Tier Automated Reminder Countdown System

When an assignment or exam duty is created with a target deadline, the ReminderScheduler (backend/app/services/reminder_scheduler.py) configures an automated 5-stage countdown:

Milestone Stage	Trigger Timing	Automated Notification Delivered to Students / Staff

7 Days Left	due_date - 7 days	■ *Your assignment '{title}' is due in 7 days on {date}. Start early!*
3 Days Left	due_date - 3 days	■ *Only 3 days left for '{title}'! Due on {date}.*
1 Day Left	due_date - 1 day	■ *'{title}' is due TOMORROW ({date})! Submit before deadline.*
Due Today	due_date (00:00)	■■ *DUE TODAY: '{title}' is DUE TODAY ({date})! Submit before 11:59 PM.*
Missed Alert	due_date + 1 day	■ *You missed the deadline for '{title}'. Contact your lecturer immediately.*

4. Background Architecture: Redis, Celery Beat & Celery Worker

• **Redis (Message Broker)**: Acts as an in-memory queue buffer. Fast HTTP requests place jobs into Redis and return instantly, keeping the UI responsive.

• **Celery Beat (The Scheduler)**: Runs continuously like an institutional clock. Every 60 seconds, it inspects target ETAs and dispatches due reminder jobs into Redis.

• **Celery Worker (The Background Processor)**: Independent worker processes listening to Redis queues. They dequeue jobs, execute heavy OCR/ML inference, write alerts to PostgreSQL, and deliver real-time notifications.

• **Unified Bell Component**: The React notification bell (UnifiedBell.jsx) queries /notifications and /reminders concurrently using Promise.allSettled, prioritizing unread alerts with role-aware click navigation.

5. Exam Duty Timetable Matrix Parser (CNN + Transformer)

HODs upload complex exam invigilation schedules containing multi-column date matrices, Forenoon/Afternoon sessions, and faculty assignments. The system uses a **Custom PyTorch Model** and a **Hybrid OCR Pipeline**:

• **CNN Backbone (ResNet18)**: Takes 9 input channels to extract low- and high-level visual features (grid lines, cell boundaries, and text regions).

• **Transformer Encoder (Self-Attention)**: Solves 2D spatial & relational reasoning. While CNNs only see local pixel neighborhoods, Transformer self-attention connects any date header cell at the top with invigilator names 10 rows below it.

• **Custom Trained Weights**: Located at backend/app/ml/table_recognition/weights/best_model.pth (**11,936,378 parameters**).

• **Multi-Session Extraction**: Extracts exact dates (e.g. *'Monday, 3 August 2026'*) and sessions (09:30-12:30 FN and 14:00-17:00 AN), maps faculty names to user accounts via fuzzy entity matching, and assigns duties automatically.

6. Placement Intelligence: spaCy, LayoutLMv3 & ATS Scoring Formula

• **spaCy NLP (en_core_web_sm)**: Extracts candidate names, universities, organizations, CGPA, and technical skills using POS tagging and noun chunking.

• **LayoutLMv3 (microsoft/layoutlmv3-base)**: Multimodal visual document model combining text tokens, page images, and 2D bounding boxes (x0, y0, x1, y1) to correctly parse complex two-column resumes.

• **ATS Scoring Formula**: Computes a score from **0 to 100%** across 6 weighted pillars:

Pillar	Weight	Detailed Evaluation Metric
1. Skills	**30%**	Quantity (6 pts/skill up to 60) + In-demand market skills (Python, AWS, React, Docker, ML up to 25) + Diversity across 5 domains (Prog, Web, Cloud, DB, AI up to 15).
2. Education	**20%**	Degree level (B.Tech / M.Tech), engineering branch alignment (CSE, ISE, ECE), and CGPA score scaling.
3. Experience	**20%**	Verified internships, work history, job role relevance, and total duration in months.
4. Projects	**15%**	Number of technical projects, modern tech stack utilization, and problem statement complexity.
5. Certifications	**10%**	Recognized credentials (AWS, Google Cloud, Coursera, NPTEL, HackerRank).
6. Formatting	**5%**	Section completeness, ATS readability, contact info, LinkedIn/GitHub links.

7. Vector Databases, Qdrant & Semantic Search (RAG)

• **Why Traditional SQL Fails**: Exact SQL keyword queries (WHERE content ILIKE '%leave%') miss synonymous phrases like *'Faculty Absence Protocol'*.

• **What is a Vector DB?**: Converts text into 384-dimensional mathematical embeddings (via all-MiniLM-L6-v2) where semantically similar concepts reside close together in geometric space measured by **Cosine Similarity**.

• **Why Qdrant?**: High-performance vector search engine written in **Rust**, implementing **HNSW (Hierarchical Navigable Small World)** graph indexing for sub-millisecond similarity retrieval.

• **What is Metadata / Payload?**: In Qdrant, each point consists of a unique id, the vector embedding, and a JSON payload (metadata: filename, department, upload date). This enables **Hybrid Search** (e.g. search for *'Python machine learning'* strictly within department = 'CSE').

• **Where Qdrant is Used**: Powering **Universal Document Search**, **Candidate-to-Job Matching**, and the **AI Chatbot Knowledge Retrieval (RAG)**.

8. Master Table of All AI Models & APIs in EduMatrix

Model / Engine	Type	Architecture / Base	Weights / Location	Purpose in Platform
Table Recognition	Custom Trained	ResNet18 CNN + 2-Layer Transformer (11.9M params)	ml/table_recognition/weights/best_model.pth	Detects rows, cols, and dates in complex exam timetables.
Intent Classifier	Custom Fine-Tuned	Sentence-BERT (SBERT) + Dense Softmax Head	ml/intent_model_sbert/	Classifies student chat messages into academic intent categories.
Sentiment Model	Fine-Tuned	DistilBERT Transformer Classifier	ml/sentiment_model/	Analyzes student feedback and institutional satisfaction.
spaCy NLP	Pre-trained	en_core_web_sm (CNN NER & POS)	spaCy Model Registry	Extracts names, universities, and technical skills from resumes.
LayoutLMv3	Pre-trained Multimodal	microsoft/layoutlmv3-base	HuggingFace Hub / Lazy Loader	Parses multi-column visual document structures in resumes.
GLiNER NER	Pre-trained Zero-Shot	urchade/gliner_multi-v2.1	services/ner_extractor.py	Extracts arbitrary entities (course codes, deadlines).
Tesseract OCR	Pre-trained OCR	LSTM Neural OCR + OpenCV CLAHE	System C++ Binary / Tesseract 5	Extracts raw text from scanned PDFs and uploaded photos.
OCR.space API	Cloud API	Cloud-based OCR Engine	services/ocr_space_service.py	External OCR fallback when local image quality is degraded.

9. Infrastructure & Architecture Rationale Summary

• **Why Docker Compose?** Orchestrates 6 isolated services (FastAPI, React Vite, PostgreSQL 15, Redis, MinIO S3, Celery Worker) with single-command deployment and environment reproducibility across Windows, Mac, and Linux.

• **Why React + Vite?** Delivers sub-second Hot Module Replacement (HMR), lightweight client state, and instantaneous virtual DOM reconciliation for live notification badges and chats.

• **Why FastAPI?** Provides asynchronous non-blocking request concurrency, strict Pydantic type validation, and auto-generated interactive OpenAPI Swagger documentation at /docs.

• **Why PostgreSQL?** Guarantees relational ACID compliance, cascading relational deletes, and strict transactional data integrity across circulars, reminders, and exam duties.